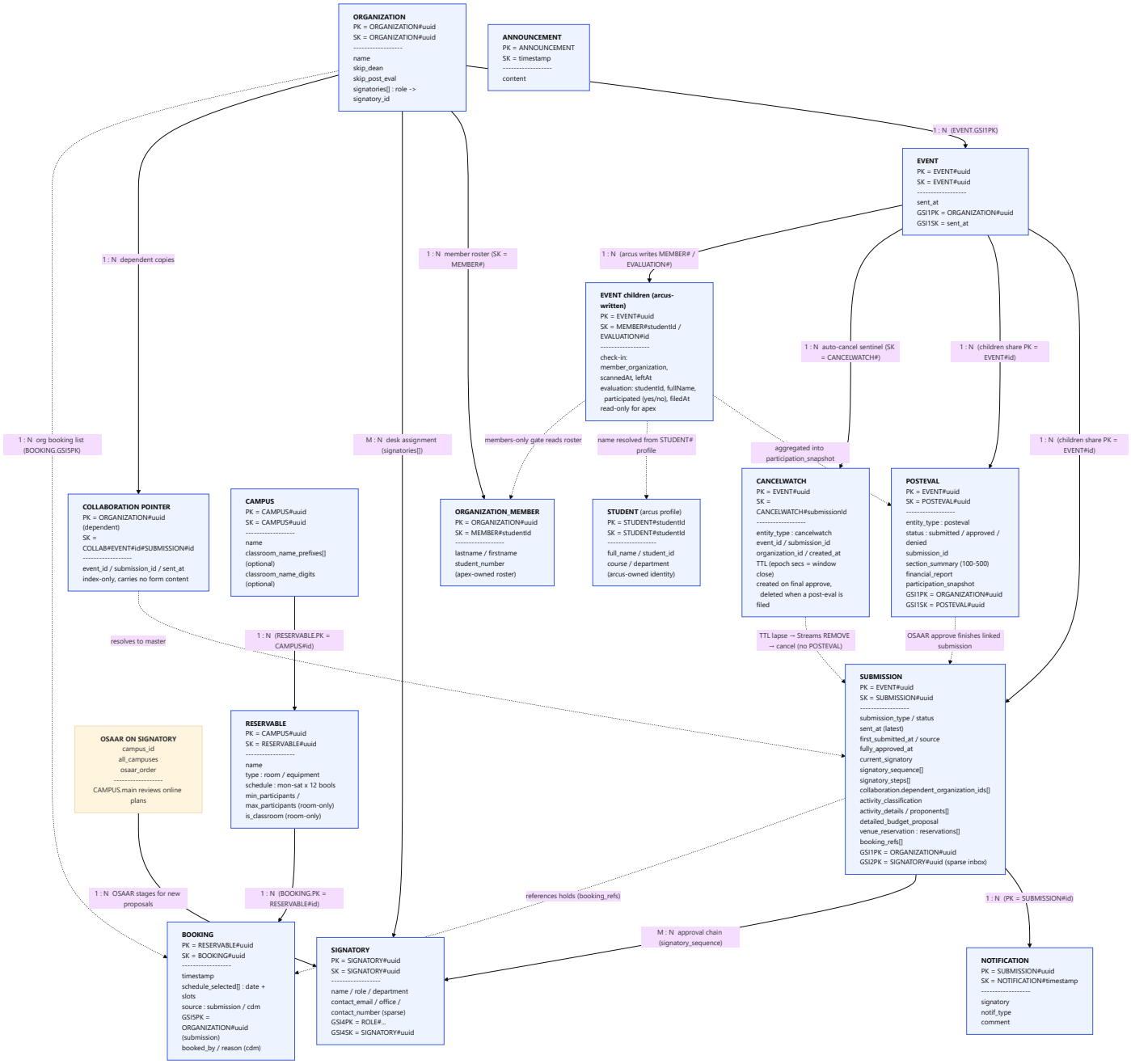
mapua-apex Data Model

DynamoDB single-table data model — conceptual ERD, object dictionary, and access patterns.

Generated 2026-11-06 01:09

Conceptual ERD



Data Objects & Properties

Business objects live in a **single DynamoDB table**. NestJS session/activity logs use the same table by default, with existing optional log-table configuration overrides. Each item is identified by a partition key (`PK`) and a sort key (`SK`); several item types share one partition so related records are read together in a single query. There are **no foreign keys and no cascade**: related IDs are *copied* onto items and resolved at write time, so deleting one item never rewrites another.

ORGANIZATION

The student organization. Also the desk directory: it records which signatory person occupies each approval role for this org.

Property	Type	Notes
`PK`	string	`ORGANIZATION#uuid`
`SK`	string	`ORGANIZATION#uuid` (same as PK)
`name`	string	Display name
`skip_dean`	boolean	Default `false` . When true, the Dean step is skipped in routing
`skip_post_eval`	boolean	Default `false` . When true, the organization cannot file a post-evaluation. Independent of `skip_dean`
`signatories`	list	Desk directory, one person per role: `{ role, signatory_id }`
`signatories[].role`	string	One of `adviser / dean / osaar / cdm / admin`
`signatories[].signatory_id`	string	Plain copy of a SIGNATORY id

Allowed desk roles are stored in the order `adviser → dean → osaar → cdm → admin` regardless of write order. `admin` may hold a desk but is **not** part of the approval sequence. `cdm` is campus-wide: routing uses the single SIGNATORY item whose role is `cdm` (`GSI4PK` `ROLE#CDM`), not a per-org desk. OSAAR stages are the OSAAR signatories assigned to the plan's campus, in `osaar_order` , then the one signatory with `all_campuses` . A per-org `osaar` desk is not used to pick the hop.

ORGANIZATION_MEMBER

The organization's member roster, **owned and managed by mapua-apex** (not by the arcus apps). One item per member, sharing the org's partition so the whole roster is read in a single query. Managed through

`/api/v1/students/members` and read by the arcus-attendance "Allow members only" scanner toggle to decide whether a scanned student may check in.

Property	Type	Notes
`PK`	string	`ORGANIZATION#uuid` (parent org partition)
`SK`	string	`MEMBER#studentId`
`lastname`	string	
`firstname`	string	
`student_number`	string	Bare student number (the `SK` without the `MEMBER#` prefix)

Created with an `attribute_not_exists(SK)` guard, so a student number is unique within the org (**409** on duplicate). Bulk CSV import writes in 25-item `BatchWriteItem` chunks and reports duplicates as skipped (`failed[]`) rather than failing the batch (max 500 rows/request). This is the authoritative membership list. It is **distinct** from the arcus STUDENT profile (arcus-owned identity, below) and **replaces** the deprecated arcus-written `ORGANIZATION` + `MEMBER` join rows (same `PK` / `SK` shape but carrying `organization_id` + `student_id` and no names), which `scripts/migrate_member_to_student.py` removes.

EVENT

A filing/event that owns one or more submissions. Its partition also holds the event's SUBMISSION children.

Property	Type	Notes
`PK`	string	`EVENT#uuid`
`SK`	string	`EVENT#uuid` (same as PK)
`sent_at`	timestamp	Creation time (UTC)
`GSI1PK`	string	`ORGANIZATION#uuid` — owning org (also identifies the *proponent* org for collaborations)
`GSI1SK`	timestamp	`sent_at` — for newest-first org event lists

SUBMISSION

A proposal paper (e.g. SAAF) filed under an event. Carries the full form body plus routing state and index keys. This is the single source of truth for a submission, including any collaboration.

Identity, routing & indexes

Property	Type	Notes
`PK`	string	`EVENT#uuid` (parent event partition)
`SK`	string	`SUBMISSION#uuid`
`submission_type`	string	e.g. `saaf`
`sent_at`	timestamp	Latest submit/resubmission time (UTC)
`first_submitted_at`	timestamp, optional	Server-owned immutable first filing; absent legacy values remain unknown
`first_submitted_at_source`	string, optional	`recorded` on new submissions or `linked_creation_evidence` from conservative backfill
`fully_approved_at`	timestamp, optional	First successful final approval, set once and preserved through completion/cancellation
`status`	string	`pending / approved / denied / returned / finished / cancelled`. `finished` is terminal and set only when OSAAR approves the event's POSTEVAL (see below). `cancelled` is terminal and has two writers: a CDM `cancel` resolution of a RESERVATION_CHANGE, and the auto-cancel DynamoDB Streams consumer when a `CANCELWATCH` sentinel's TTL lapses with nothing filed (see below)
`cancelled_at`	timestamp	Optional — set only by the auto-cancel stream consumer (the CDM cancel path does not stamp it)
`current_signatory`	string	`SIGNATORY#uuid` holding the paper now
`signatory_sequence`	list	Ordered `SIGNATORY#uuid` routing snapshot, written at submit time
`signatory_steps`	list	Same snapshot with `{ key, role, signatory_id, organization_id }`. Keys: `adviser:ORG#id`, `dean:ORG#id`, `osaar_1` .. `n`, `cdm`. Absent on items sent before stages were stored; those items are not backfilled
`collaboration.dependent_organization_ids`	list	`ORGANIZATION#uuid` dependents; empty/absent = no collaboration

Property	Type	Notes
`GSI1PK`	string	`ORGANIZATION#uuid` (proponent org)
`GSI1SK`	string	`SUBMISSION#uuid`
`GSI2PK`	string	`SIGNATORY#uuid` — sparse signatory inbox (present while `pending` / `returned`)
`GSI2SK`	timestamp	Desk queue time

The NestJS `/api/v2` report API enumerates the authorized organization/collaboration scope before applying dates; GSI1 is organization/ID-based, not date-based. Editable updates condition on status/latest date/current desk and do not overwrite reporting timestamps from stale reads. Final approval uses `if_not_exists`. Historical recovery is dry-run-first, accepts only reliable linked creation/final-approval evidence, and conditionally avoids replacing concurrent values. Unknown dates and log retention produce explicit coverage notes, never fabricated cohort dates. Reports do not create persistent data objects.

activity_classification

Property	Type	Notes
`activity_type`	string	`co-curricular / extra-curricular`
`total_org_members`	number	
`nature`	string	`major / minor` — set by OSAAR via the classification PATCH, not by the student create body

proponents[] (list of people)

`id`, `position_title`, `first_name`, `middle_name`, `last_name`, `suffix`, `student_number`, `program_and_year`, `date_of_submission`, `department`, `position_of_applicant`, `org_or_course_section`, `contact_number`, `email_address`, `facebook_link`.

activity_details

`title_and_nature`, `description`, `objectives`, `venue`, `date_of_event`, `day_of_event`, `time_of_event`, `expected_participants`, `individual_contribution`, `proposed_budget`. `time_of_event` is 24-hour `"HH:MM – HH:MM"`; saved times fall between 7:00 AM and 9:00 PM, end not earlier than start, and start ≠ end.

institutional_alignment

`mission_statements { competitive, research, solutions }`, `core_values_explanation`, `peo_explanation`, `sdg_explanation`.

detailed_budget_proposal

`has_budget`, `items[] { item_no, unit, quantity, price_per_unit, total }`, `grand_total`.

Property	Type	Notes
`has_budget`	boolean	Drives whether the budget UI is shown; a **missing** key means an older paper that had a budget (`true`). When `false` the paper stores `items: []`, `grand_total: 0` and `activity_details.proposed_budget` / `individual_contribution` of `0`; readers hide the budget and the post-eval wizard skips its financial step. Does **not** affect routing (only `has_reservation` does)

venue_reservation & booking_refs

Property	Type	Notes
`has_reservation`	boolean	Drives whether the CDM desk joins the routing chain
`reservations[]`	list	One entry per picked reservable: `{ reservable_id, campus_id, name, type, selections[], remarks }`
`reservations[].type`	string	`room` | `equipment`
`reservations[].selections[]`	list	`{ date: "YYYY-MM-DD", slots: [int 0-11] }` — 0-based indices into the 07:00–21:00 / 70-min day (12 slots)
`booking_refs[]`	list	Sparse reverse index `{ pk: "RESERVABLE#uuid", sk: "BOOKING#uuid" }`; present only while the paper holds BOOKINGs. Released + removed on deny/return/cancel-resolution, reconciled on re-edit and on a reschedule-resolution

Legacy items stored before this shape may still carry `equipment_requested` / `function_rooms` / `audiovisual_equipment`; reads tolerate both. On submit each selection is validated against the reservable's weekly template and existing approved BOOKINGs, then written as `source:"submission"` holds with `approval_status: "pending"` — a slot taken in the meantime aborts with **409** and persists nothing. Pending holds do not occupy slots until the CDM desk approves the paper (see BOOKING).

NOTIFICATION

One row in a submission's approval/timeline history. Stored in a partition keyed by the submission id (a separate partition from the event).

Property	Type	Notes
`PK`	string	`SUBMISSION#uuid`
`SK`	string	`NOTIFICATION#timestamp` (identity; UTC second precision)
`signatory`	string	`SIGNATORY#uuid` author (snapshot)
`notif_type`	string	`approved / fully approved / denied / returned / event scheduled / event auto-cancelled / reservation change requested / reservation cancelled / reservation rescheduled / reservation change denied` — the four `reservation ...` types are system-only (written by RequestReservationChange / ResolveReservationChange, never client-postable); `event scheduled` (final approval) and `event auto-cancelled` (stream consumer) are also system-only with an empty signatory
`comment`	string	Mandatory when `denied` or `returned`

Approve / return / deny insert a NOTIFICATION row as a side effect. A PUT replaces `notif_type` / `comment` in place and does **not** change the SK.

SIGNATORY

A person (the signatory identity). Has **no** `organization_id`; desk assignment lives on `ORGANIZATION.signatories`. Which Cognito user acts as this person is `custom:signatory_id` on the user pool, not a field on this item. `PUT /admins/signatories/{id}/account` maintains it. An organization account uses `custom:organization_id` the same way.

Property	Type	Notes
`PK`	string	`SIGNATORY#uuid`
`SK`	string	`SIGNATORY#uuid` (same as PK)

Property	Type	Notes
`name`	string	
`role`	string	`adviser / cdm / dean / osaar / admin`
`department`	string	Deans only (e.g. `SOIT`)
`campus_id`	string	OSAAR only. Bare campus uuid. Many OSAAR accounts may share one campus
`all_campuses`	boolean	OSAAR only. True for the single account that approves every plan. No `campus_id`
`osaar_order`	number	OSAAR only. 1-based order among that campus's accounts
`contact_email`	string	Sparse. Desk email a proponent may write to; may differ from the Cognito login
`office`	string	Sparse. Room or desk where this person can be found in person
`contact_number`	string	Sparse. Desk phone
`GSI4PK`	string	`ROLE#ADVISER / ROLE#CDM / ROLE#DEAN / ROLE#DEAN#{DEPARTMENT} / ROLE#OSAAR / ROLE#ADMIN`
`GSI4SK`	string	`SIGNATORY#uuid`

The three contact attributes are written only when non-blank after trimming, so clearing one on `PUT /admins/signatories/{id}` removes the attribute instead of storing an empty string. They reach a proponent solely through the derived `approval_chain_contacts` array on `GET /students/events/{id}/submissions/{id}` (one `BatchGetItem` over the chain's SIGNATORY rows); no submission list, reviewer detail, or admin detail carries them.

Other items copy the signatory id rather than join back to SIGNATORY: `ORGANIZATION.signatories[].signatory_id`, `SUBMISSION.current_signatory` / `GSI2PK`, and `NOTIFICATION.signatory`. Renaming keeps the same uuid; deleting a SIGNATORY does not rewrite those copies. CDM stays one person. Several OSAAR people may share a campus. New proposals take that campus's accounts in `osaar_order`, then the all-campus account.

APPROVAL SETTINGS

Legacy item (`PK` / `SK` `SETTINGS#APPROVAL`). Routing no longer reads it. OSAAR order is `osaar_order` on each campus OSAAR signatory. `PUT /admins/approval-settings` rewrites that order for one campus. Proposals already sent keep the stages stored on their own `signatory_steps`.

COLLABORATION POINTER

An index-only item, one per dependent organization, that makes the proponent's submission discoverable in a dependent's dashboard. It carries **no** form content — the master SUBMISSION stays the single source of truth.

Property	Type	Notes
`PK`	string	`ORGANIZATION#uuid` — the **dependent** org (not the proponent)
`SK`	string	`COLLAB#EVENT#uuid#SUBMISSION#uuid`
`event_id`	string	`EVENT#uuid`
`submission_id`	string	`SUBMISSION#uuid`
`sent_at`	timestamp	

Queried from the dependent's partition (`SK` begins `COLLAB#`); no GSI is needed because DynamoDB cannot index the N values of an array. A dependent may read the shared submission and its NOTIFICATION timeline but cannot edit or resubmit — only the proponent may.

POSTEVAL

A post-evaluation filed by the org submitter after an approved event is held, within the 3-day evaluation window. It lives in the **event's** partition (alongside its SUBMISSION) and is discovered per-org through GSI1. OSAAR approval is the **only** path that transitions the linked submission `approved → finished` .

Property	Type	Notes
`PK`	string	`EVENT#uuid` (parent event partition)
`SK`	string	`POSTEVAL#uuid`
`entity_type`	string	`"posteval"`
`organization_id`	string	Bare org uuid
`event_id`	string	Bare event uuid
`submission_id`	string	Bare uuid of the linked approved paper
`status`	string	`submitted / approved / denied`
`submitted_at`	timestamp	
`approved_at`	timestamp	Sparse — stamped on OSAAR approve
`denied_at`	timestamp	Sparse — stamped on OSAAR deny

Property	Type	Notes
`event_title`	string	Snapshot of `activity_details.title_and_nature`
`section_summary`	string	100–500 chars — how the event went
`financial_report`	object	`{ budget_items[], added_items[], initial_grand_total, actual_grand_total }`; every amount capped at 5 digits (0–99999)
`participation_snapshot`	object	`{ attendance_count, attendance[], evaluation_count, evaluations[] }` — frozen at write time
`GSI1PK`	string	`ORGANIZATION#uuid`
`GSI1SK`	string	`POSTEVAL#uuid` (mirrors SUBMISSION for org discovery)

`financial_report.budget_items[]` are `{ item, unit, quantity, budgeted_total, actual_amount }`; `added_items[]` are `{ item, unit, quantity, price_per_unit, actual_amount }`. `participation_snapshot.attendance[]` are `{ student_id, full_name, department, member_organization, scanned_at, left_at }` and `evaluations[]` are `{ evaluation_id, student_id, full_name, participated, filed_at }`.

Write gates (`WritePostEval`): the linked SUBMISSION must be `approved` / `finished` (**422** otherwise), `EventWindow::EVALUATION` must still be open (**422** — "window is closed"), and no `submitted` / `approved` POSTEVAL may already exist for the event (**409**). The snapshot is aggregated from the event's `MEMBER#` / `EVALUATION#` children (below) at write time and stored with an `attribute_not_exists(PK)` guard on the SK.

OSAAR actions: `ApprovePostEval` sets `status = approved` + `approved_at`, then moves the linked SUBMISSION `approved → finished` (+ `finished_at`) — both idempotent, and the window is **not** re-checked (approval may follow the window closing). `DenyPostEval` sets `status = denied` + `denied_at` and leaves the submission `approved`. Approving a denied post-eval (or denying an approved one) aborts **422**.

CANCELWATCH

The time-to-live sentinel behind the post-evaluation **auto-cancel**. One item lives in the **event's** partition per approved paper (alongside its SUBMISSION and POSTEVAL children). Its expiry is what fires the auto-cancel.

Property	Type	Notes
`PK`	string	`EVENT#uuid` (parent event partition)

Property	Type	Notes
`SK`	string	`CANCELWATCH#uuid` (the linked bare SUBMISSION uuid)
`entity_type`	string	`cancelwatch`
`event_id`	string	Bare event uuid
`submission_id`	string	Bare uuid of the linked approved paper
`organization_id`	string	Bare org uuid
`created_at`	timestamp	Final approval time (UTC)
`TTL`	number	Unix epoch seconds = the evaluation window close (`EventWindow::EVALUATION` for `activity_details.date_of_event` , `ARCUS_POST_EVALUATION_WINDOW_DAYS` , Asia/Manila). The table's single uppercase `TTL` attribute

Created on the final approve of a paper whose org does **not** `skip_post_eval` (a `skip_post_eval` org files nothing, so it must never auto-cancel; a paper with an unparseable `date_of_event` gets no sentinel). **Deleted** immediately when a post-eval is filed (the belt). When the TTL lapses DynamoDB expires the item and emits a **Streams** `REMOVE` that a consumer Lambda handles: it re-queries the partition for a `POSTEVAL#` child (never trusting cross-key ordering) and, finding none, cancels the SUBMISSION through a conditional update guarded on `status = approved` (a finished/denied/already-cancelled paper or a replayed `REMOVE` no-ops), then writes an `event auto-cancelled` NOTIFICATION and emails the student. TTL expiry is best-effort and can lag by hours — harmless against a 3-day deadline. The whole feature is behind `ARCUS_AUTO_CANCEL_ENABLED` (default on): when off, approve writes no sentinel and `evaluateAndCancel` no-ops.

EVENT-partition children (written by the arcus companion apps)

The `arcus-attendance` and `arcus-evaluation` microservices share the **same** physical table and write into the apex `EVENT#uuid` partition, keyed by the apex `event_id` . apex never writes these — it only reads them (via `AggregateEventParticipants`) to build a POSTEVAL's `participation_snapshot` . Attendance uses the event-day window. Participant evaluations require the signed-in student's check-in and use the evaluation window (event day through three days afterward by default, in Asia/Manila).

Object	PK	SK	Attributes
Attendance check-in	`EVENT#uuid`	`MEMBER#{studentId}`	`member_organization` , `scannedAt` , `leftAt` (sparse)

Object	PK	SK	Attributes
Participant evaluation	`EVENT#uuid`	`EVALUATION#{evaluationId}`	`evaluationId`, `studentId`, `fullName`, `participated` (`"yes"` / `"no"`), `filedAt`, `eventId`

Arcus Evaluation also writes full response copies at `PK = SK = EVALUATION#{id}` and `PK = EVALUATION`, `SK = {filedAt}#{id}`. These retain the identity/event metadata plus `schemaVersion: 2`, `eventTitle`, all applicable `ratings` (a numeric map for 23 criteria), `discoverySources` (boolean map), `comments`, and `othersSpecify`. Both full copies and the slim event child are committed in one DynamoDB transaction. No new S3 object or bucket/key attribute is written; the browser fills the original FM-SA-05-01 template for preview and export. Legacy metadata without full responses remains visible but cannot regenerate a PDF. No legacy object migration or deletion is performed.

The Arcus officer panel reads live participants and the active POSTEVAL, falling back to the latest denied filing by `submitted_at`. Its `submission_id` resolves the associated paper; orphaned filings are ignored. These reads do not change review status or replace the frozen `participation_snapshot`.

The scanner also keeps a student **profile** item at `PK = SK = STUDENT#{studentId}` holding `student_id`, `full_name`, `course`, `department` (renamed from the former `MEMBER#` profile; the EVENT check-in `SK` above stays `MEMBER#{studentId}` because apex splits an event partition's children by `SK` prefix). apex resolves each check-in's name by reading the `STUDENT#{studentId}` profile. apex reads these camelCase attributes (with snake_case fallbacks) and normalizes `participated` to a boolean in the snapshot.

The "Allow members only" gate reads the apex-owned **ORGANIZATION_MEMBER** roster (`PK = ORGANIZATION#uuid`, `SK = MEMBER#{studentId}`), **not** this profile. arcus registration no longer writes any `ORGANIZATION` + `MEMBER` join row; the former ones are deprecated and removed by `scripts/migrate_member_to_student.py`.

ANNOUNCEMENT

A campus-wide announcement. All announcements share one partition; the SK (timestamp) is the identity. Admin Cognito group only.

Property	Type	Notes
`PK`	string	`ANNOUNCEMENT` (constant)
`SK`	string	`timestamp` — identity (`sent_at` in the API); newest-first list
`content`	string	

A PUT replaces `content` in place and does not change the SK.

CAMPUS

A reservable campus (e.g. `Intramuros Campus`, `Makati Campus`), seeded by osaar on the `/campus` page. The SAAF reservation step and the `activity_details.venue` dropdown read the list.

Property	Type	Notes
`PK`	string	`CAMPUS#uuid`
`SK`	string	`CAMPUS#uuid` (same as PK)
`name`	string	Display name
`main`	boolean	True for the single campus whose OSAARs review online events. A second `true` is rejected and the flag is not moved
`classroom_name_prefixes`	list	**Optional.** Allowed classroom-name prefix tokens (1–10 letters each, up to 30; normalized uppercase + deduped). Stored only when paired with `classroom_name_digits`
`classroom_name_digits`	number	**Optional.** Required digit count (1–4). With the prefix list it defines how this campus's classroom rooms are named (e.g. `MPO` + 3 → `MPO123`)

List scans `PK` begins `CAMPUS#` AND `PK = SK`. Delete is guarded: **409** if any RESERVABLE still lives under `PK = CAMPUS#id`, or if an OSAAR signatory still stores this campus id. Rename in place (PUT) to keep the same uuid so RESERVABLE children and stored booking `campus_id` snapshots stay valid. The classroom format is authored on the osaar `/campus` Add/Edit page; a campus without it cannot host classroom rooms.

RESERVABLE

A bookable room or equipment item under a campus. Its `SK` is globally unique and doubles as the BOOKING partition key.

Property	Type	Notes
`PK`	string	`CAMPUS#uuid` (parent campus)
`SK`	string	`RESERVABLE#uuid`
`name`	string	
`type`	string	`room` | `equipment`

Property	Type	Notes
`schedule`	object	Recurring weekly template: `monday` .. `saturday`, each exactly 12 booleans (one per 07:00–21:00 / 70-min slot; `true` = available). Sunday is never reservable; an absent day is all-unavailable
`min_participants`	number	**Room-only, optional.** Lower bound on the expected headcount a room can hold. Omitted/unset = no lower limit; never stored for `equipment`
`max_participants`	number	**Room-only, optional.** Upper bound on the expected headcount a room can hold. Omitted/unset = no upper limit; never stored for `equipment`
`is_classroom`	boolean	**Room-only, optional.** True marks the room as a classroom whose `name` must match the owning campus's classroom format. Stored only when `type` is `room` and the flag is true; read back as false otherwise

Added/edited by cdm on the `/reservables` page (CSV for name+type, a grid for the schedule; participant bounds are room-only inputs, disabled and dropped for equipment). The `is_classroom` toggle is room-only and shown-but-disabled until the selected campus defines a classroom format; when enabled the room `name` is validated against `<one prefix><exactly N digits>` at create/update (**422** on the name if it misses, or **422** on `is_classroom` if the campus has no format). On SAAF submit the activity's `expected_participants` is checked against each picked room's bounds (`BookingRecords::assertAvailable`); an out-of-range headcount aborts with **422** before any BOOKING is written. Delete is guarded: **409** if any BOOKING still occupies `PK = RESERVABLE#id`.

BOOKING

A concrete hold on a reservable's slots for specific dates. One BOOKING per (reservable, submission) or per (reservable, CDM reserve action). Two flavors share the item type, distinguished by `source`.

Property	Type	Notes
`PK`	string	`RESERVABLE#uuid`
`SK`	string	`BOOKING#uuid`
`timestamp`	timestamp	Create time (UTC)
`schedule_selected`	list	`[{ date: "YYYY-MM-DD", slots: [int 0-11] }]` (normalized)

Property	Type	Notes
`source`	string	`submission` (SAAF-created hold) | `cdm` (manual CDM hold)
`campus_id`	string	Bare campus uuid (snapshot)
`reservable_name`	string	Snapshot at write time
`reservable_type`	string	`room` | `equipment` (snapshot)

Submission-sourced (`source: "submission"`) adds GSI5 + traceability, and is released on deny/return/re-edit via the SUBMISSION's `booking_refs` :

Property	Type	Notes
`GSI5PK`	string	`ORGANIZATION#uuid` — owning org (org booking list)
`GSI5SK`	timestamp	`timestamp`
`event_id` / `submission_id` / `organization_id`	string	Bare uuids for traceability and read-only CDM labelling
`approval_status`	string	`pending` (written on SAAF submit/re-edit) | `approved` . Absent on legacy items, which read as approved
`approved_at`	timestamp	Sparse — stamped when CDM approves the paper
`approved_by`	string	Sparse — `SIGNATORY#uuid` of the approving CDM signatory

Only **approved** holds occupy slots. On the paper's last approval (the CDM desk, which is always last when a venue is reserved) `ApproveSubmission` re-checks every `booking_refs` hold against the approved bookings — a slot taken in the meantime aborts **409** with nothing written, leaving the paper on the CDM desk to return or deny — then flips each hold to `approved` . The CDM review screen reads the same check (without writing) from `GET /signatories/events/{event}/submissions/{submission}/booking-conflicts` . Pending holds are still listed on the CDM Reserve/View page (labelled) and still block reservable deletion. Legacy holds are stamped by `scripts/backfill_booking_approval.py` .

Manual (`source: "cdm"`) carries **no** GSI5 and **no** submission linkage (so it never surfaces in an org's list); it is released only by CDM delete:

Property	Type	Notes
`booked_by`	string	`SIGNATORY#uuid` (the CDM signatory)
`reason`	string	Optional purpose note (omitted when blank)

Both flavors share one availability engine: a slot is free only when the weekly template allows it AND no approved BOOKING (either source; manual holds always count) occupies that date+slot. Conflict detection runs on the base RESERVABLE partition — never GSI5 — so it works before the (operator-provisioned) organization index exists.

A submission-sourced hold is also **edited in place** through a RESERVATION_CHANGE resolution: a same-reservable date/slot change patches `schedule_selected` on the existing `PK` / `SK` (keys, GSI5 linkage and `submission_id` / `event_id` / `organization_id` all survive, and the SUBMISSION's `booking_refs` entry stays valid); a room swap deletes the old BOOKING and recreates a `source:"submission"` hold under the new RESERVABLE partition, swapping the `booking_refs` entry to the new keys in the same submission update. The recreated hold is written `approval_status: "approved"` (the paper is already approved).

RESERVATION_CHANGE

A student's request to cancel or reschedule the holds on an **approved** paper (which the student cannot edit directly), resolved by CDM. Each request freezes the concrete BOOKING records it targets and keeps a full before/after audit trail once resolved.

Property	Type	Notes
`PK`	string	`EVENT#uuid` (parent event partition)
`SK`	string	`RESERVATION_CHANGE#uuid`
`item_type`	string	`reservation_change`
`organization_id` / `submission_id` / `campus_id`	string	Bare uuids, snapshotted at request time
`requested_by`	string	`STUDENT#<sub>` from the JWT
`requested_at`	timestamp	
`request_type`	string	`cancel` | `reschedule`
`reason`	string	Required (≤500)
`suggested_date`	string	**Sparse, reschedule only.** `"YYYY-MM-DD"`, advisory — CDM is not bound by it
`bookings[]`	list	The targeted holds, **frozen** from the paper's live `booking_refs` at request time: `{ pk, sk, reservable_id, reservable_name, reservable_type, schedule_selected[] }`
`status`	string	`pending / resolved / denied`
`GSI6PK`	string	`CAMPUS#uuid` — CDM per-campus queue

Property	Type	Notes
`GSI6SK`	string	`status#requested_at` (`pending# ...` , `resolved# ...` , `denied# ...`)

On resolution the item also carries `decision` (`cancel / reschedule / deny`), `resolved_by` (`SIGNATORY#uuid`), `resolved_at` , `comment` , and an `applied` audit object:

Property	Type	Notes
`applied.bookings[]`	list	One entry per targeted hold: `{ pk, sk, action: released | kept | updated | replaced, before: { reservable_id, schedule_selected }, after: {…} | null, new_pk, new_sk }` — `new_pk` / `new_sk` only on a room swap (point at the recreated BOOKING)
`applied.submission`	object | null	`{ status_before, status_after, date_of_event_before/after, time_of_event_before/after, booking_refs_before, booking_refs_after }` ; `null` for a deny

`applied` is an audit trail only — never read back as the source of truth. One pending request per paper. The student side lists the full history with a base-table query on `PK = EVENT#uuid` , `SK` begins `RESERVATION_CHANGE#` (no GSI); the CDM queue reads **GSI6** (operator-provisioned in AWS, mirroring GSI5; the CDM list endpoint fails loudly until it exists). Resolve is guarded: the request must still be `pending` (`finalize` uses a conditional put on `attribute_not_exists(resolved_at)` so two CDM users cannot resolve it twice), the paper must still be `approved` , and every snapshotted hold must still appear in the paper's live `booking_refs` (a stale request aborts **409** with zero writes). The request window closes at the end of `activity_details.date_of_event` in Asia/Manila; CDM may resolve at any time.

mapua-apex — DynamoDB Access Patterns

DynamoDB is modeled around **access patterns**, not normalized relations: every query is designed first, then the partition key (PK), sort key (SK), and global secondary indexes (GSI) are chosen to serve it. Business patterns hit a **single table**; session/activity logs use that table by default with optional existing log-table overrides. There are no joins — related IDs are copied onto items at write time.

Index summary:

Index	Key	On	Purpose
Base table	`PK` / `SK`	all items	Primary access
GSI1	`GSI1PK` / `GSI1SK`	EVENT, SUBMISSION, POSTEVAL	Org event walk (deadlines), student & admin submission lists, org/OSAAR post-eval list
GSI2	`GSI2PK` / `GSI2SK`	SUBMISSION	Sparse signatory inbox (`pending` / `returned`)
GSI4	`GSI4PK` / `GSI4SK`	SIGNATORY	Role directory (`ROLE# ...`)
GSI5	`GSI5PK` / `GSI5SK`	BOOKING	Org-scoped booking list (submission-sourced holds only; operator-provisioned)
GSI6	`GSI6PK` / `GSI6SK`	RESERVATION_CHANGE	CDM per-campus reservation-change queue (`status#requested_at` ; operator-provisioned)

Core access patterns

#	Access pattern	Operation	Key condition	Index
1	Org event walk (deadlines list)	Query	`GSI1PK = ORGANIZATION#id` (first 25 EVENT items, oldest first)	GSI1
2	Event children (its deadlines)	Query	`PK = EVENT#id` , `SK` begins `DEADLINE#`	Base
3	Student's own submissions	Query	`GSI1PK = ORGANIZATION#id` , `GSI1SK` begins `SUBMISSION#` (all papers, paginated)	GSI1

#	Access pattern	Operation	Key condition	Index
4	Collaboration submissions (dependent view)	Query	`PK = ORGANIZATION#<dep>`, `SK` begins `COLLAB#`, then resolve each pointer to its master SUBMISSION	Base
5	Single submission detail	GetItem → BatchGetItem	`PK = EVENT#id`, `SK = SUBMISSION#id`; the student detail then batch-reads the chain's `SIGNATORY#id` rows to derive `approval_chain_contacts`	Base
6	Submission timeline / audit	Query	`PK = SUBMISSION#id`, `SK` begins `NOTIFICATION#`	Base
7	Signatory inbox (desk queue)	Query	`GSI2PK = SIGNATORY#id`, `GSI2SK` by time (sparse)	GSI2
8	Signatory directory by role	Query	`GSI4PK = ROLE#...`, `GSI4SK` begins `SIGNATORY#`	GSI4
8b	OSAAR approval stages	Query	`GSI4PK = ROLE#OSAAR`, then filter `campus_id` and sort `osaar_order`. The all-campus account is the row with `all_campuses`. Online plans use the `CAMPUS` item with `main`	GSI4
9	Current signatory profile (`/signatories/me`)	GetItem	`PK = SIGNATORY#id`, `SK = SIGNATORY#id`	Base

#	Access pattern	Operation	Key condition	Index
10	Admin: all submissions	Query	per org `GSI1PK = ORGANIZATION#id`, `GSI1SK` begins `SUBMISSION#`	GSI1
11	Student org record (`/students/organization`)	GetItem	`PK = ORGANIZATION#id`, `SK = ORGANIZATION#id`	Base
12	Org directory (`/students/organizations`)	Scan (fully paginated)	`PK` begins `ORGANIZATION#`, `PK = SK`; org list → `{ organization_id, name }`	Base
13	Announcement list	Query	`PK = ANNOUNCEMENT`, `SK` desc (newest first)	Base
14	Announcement show / edit / delete	GetItem / PutItem / DeleteItem	`PK = ANNOUNCEMENT`, `SK = timestamp`	Base
15	Write submission (+ collab pointers)	PutItem	`PK = EVENT#id`, `SK = SUBMISSION#id`; maintain one COLLAB pointer per dependent	Base
16	Approve / return / deny	UpdateItem	advance `current_signatory` / `GSI2PK` from stored `signatory_sequence`; write NOTIFICATION	Base
17	Admin: delete organization (guarded)	Query → DeleteItem	block (**409**) if `GSI1PK = ORGANIZATION#id` has events/submissions or base `PK = ORGANIZATION#id`, `SK` begins `COLLAB#`; else DeleteItem `PK = SK = ORGANIZATION#id`	GSI1 + Base

#	Access pattern	Operation	Key condition	Index
18	Admin: delete signatory (guarded)	Scan/Query → DeleteItem	block (**409**) if any `ORGANIZATION.signatories` desk holds the id or `GSI2PK = SIGNATORY#id` (open inbox); else DeleteItem `PK = SK = SIGNATORY#id`	GSI2 + Base
19	Campus list (osaar `/campus`, SAAF venue dropdown)	Scan	`PK` begins `CAMPUS#` AND `PK = SK`	Base
20	Reservables under a campus	Query	`PK = CAMPUS#id`, `SK` begins `RESERVABLE#`	Base
21	Reservable availability (template + per-date free/booked slots)	Query	`PK = RESERVABLE#id`, `SK` begins `BOOKING#`; overlay approved bookings (pending SAAF holds skipped) on the weekly template across the window	Base
22	CDM View/Reserve calendar (raw bookings, both sources)	Query	`PK = RESERVABLE#id`, `SK` begins `BOOKING#` (filtered to dates in window)	Base
23	Create booking (submission hold or CDM manual)	Query → PutItem	validate selections vs template + existing approved bookings (**409** on conflict) and, on submission, expected headcount vs each room's `min/max_participants` (**422** out of range), then PutItem `PK = RESERVABLE#id`, `SK = BOOKING#id`	Base
24	Release holds on deny / return / re-edit / cancel-resolution	DeleteItem	delete each `booking_refs[] {pk, sk}` (submission holds)	Base

#	Access pattern	Operation	Key condition	Index
25	Delete a manual booking (CDM)	GetItem → DeleteItem	**409** if `source` is `submission`; else DeleteItem `PK = RESERVABLE#id`, `SK = BOOKING#id`	Base
26	Admin: delete campus / reservable (guarded)	Query → DeleteItem	block (**409**) if a campus owns any RESERVABLE, or a reservable owns any BOOKING; else DeleteItem	Base
27	Create / edit reservable (CDM Add/Edit)	GetItem → PutItem	for a `room` flagged `is_classroom`, GetItem the owning CAMPUS and enforce it defines a classroom format (**422** on `is_classroom` if not) and the `name` matches `<prefix> <N digits>` (**422** on `name`), then PutItem `PK = CAMPUS#id`, `SK = RESERVABLE#id`; room-only `min/max_participants` + `is_classroom` dropped for `equipment`	Base
28	Eligible post-eval events (student step 1)	Query	per approved/finished submission compute the effective date (booking date when reserved, else `date_of_event`), keep those already passed with `EventWindow::EVALUATION` open, minus events that already have a `submitted` / `approved` POSTEVAL	GSI1 + Base

#	Access pattern	Operation	Key condition	Index
29	Org post-eval list (student dashboard)	Query	`GSI1PK = ORGANIZATION#id`, `GSI1SK` begins `POSTEVAL#`	GSI1
30	OSAAR post-eval queue (cross-org)	Query	per org `GSI1PK = ORGANIZATION#id`, `GSI1SK` begins `POSTEVAL#` (optional `status` filter)	GSI1
31	Single post-eval detail	GetItem	`PK = EVENT#id`, `SK = POSTEVAL#id`	Base
32	Aggregate event participants (post-eval Section 3)	Query	`PK = EVENT#id`; split `SK` begins `MEMBER#` (attendance; resolve names via `PK = SK = STUDENT#{studentId}`) vs `EVALUATION#`	Base
33	Write post-eval (gated)	Query → PutItem	enforce submission approved/finished + window open (**422**) + no active POSTEVAL (**409**), freeze the snapshot, PutItem `PK = EVENT#id`, `SK = POSTEVAL#id` (`attribute_not_exists(PK)`)	Base + GSI1
34	Approve / deny post-eval (OSAAR)	UpdateItem	POSTEVAL `submitted → approved` / `denied`; on approve also SUBMISSION `approved → finished` (idempotent)	Base

#	Access pattern	Operation	Key condition	Index
35	Arcus student: attended evaluatable events	Query → BatchGetItem	List published evaluation-window submissions across orgs, then check `PK = EVENT#id`, `SK = MEMBER#{ verifiedStudentId}` in deduplicated batches of 100; retry unprocessed keys, fail on exhaustion	GSI1 + Base
36	Arcus officer: live participants + review status	Query	`PK = EVENT#id`, prefixes `SUBMISSION#` and `POSTEVAL#`; prefer active/latest filing and its linked submission, enforce org scope, then aggregate participants (32)	Base
37	Arcus student: file full response	TransactWriteItems	Full detail `PK = SK = EVALUATION#id`; full list `PK = EVALUATION`, `SK = filedAt#id`; slim `EVENT#id/EVALUATION#id` child, all conditional puts	Base
38	Arcus officer: event-scoped export	Query → BatchGetItem	Query `EVENT#id`, `SK` begins `EVALUATION#`, then batch-read full `EVALUATION#id` detail records; filter/sort, return up to 100; PDF generation stays in browser	Base

#	Access pattern	Operation	Key condition	Index
39	CDM approval of a paper's holds	GetItem → UpdateItem	on the last (CDM) approve, GetItem each `booking_refs[]` hold + its RESERVABLE, re-check against approved bookings (**409**, nothing written, on conflict), then UpdateItem `approval_status = approved`, `approved_at`, `approved_by`; the same check (no writes) backs `GET …/booking-conflicts`	Base
40	Create / delete the auto-cancel sentinel	PutItem / DeleteItem	final approve (non-`skip_post_eval` org) PutItem `PK = EVENT#id`, `SK = CANCELWATCH#id` with `TTL` = window close; filing a post-eval DeleteItem the same key. Gated by `ARCUS_AUTO_CANCEL_ENABLED` (default on)	Base

#	Access pattern	Operation	Key condition	Index
41	Auto-cancel on window lapse (stream consumer)	Query → UpdateItem → PutItem	the sentinel's TTL expiry emits a Streams `REMOVE`; re-Query `PK = EVENT#id`, `SK` begins `POSTEVAL#` (a filed post-eval no-ops), then UpdateItem the SUBMISSION `approved → cancelled` + `cancelled_at` guarded on `status = approved` (a finished/denied/cancelled paper or replayed `REMOVE` no-ops), PutItem an `event auto-cancelled` NOTIFICATION, and email the student	Base

---

NestJS reporting access patterns

- Session logs: monthly GSI1 `LOG#SESSION#YYYY-MM` partitions; activity logs: daily GSI3 `DATE#YYYY-MM-DD` partitions. Inclusive Manila days traverse every intersecting UTC bucket. Opaque continuation includes bucket/key, filter/caller fingerprint, direction and retrieval cutoff. Empty filtered pages do not terminate retrieval.
- Proposal reports reuse patterns 3/4/10, enumerating the fully paginated organization directory first. GSI1 is not date-indexed; date filtering/aggregation occurs after authorized traversal. Compound event/submission identity deduplicates collaboration pointers.
- Post-evaluation reports reuse 29/30; campus reservation changes query `GSI6PK = CAMPUS#id` and hydrate related submissions in batches. Batch reads deduplicate and retry unprocessed keys; missing/throttled required records fail rather than silently reduce totals.
- New submission first-filing/source fields are preserved by conditional editable updates. Final approval sets `fully_approved_at` only if absent. Historical backfill is offline-testable and dry-run-first; cloud runs require approval.
- Report POSTs are read-only HTTP 200/no-store projections, bounded by 366 days, 10,000 detail rows, 4 MiB, and 20 seconds. Summary-only aggregation is complete within those budgets. Unknown dates and retention coverage are disclosed separately from retrieval success. No report storage, new GSI, or new visit collection is introduced. Report generation is role-gated: the session report requires `session_admin` (admin, super_admin) and the four collection reports require `report_admin` (admin, osaar, super_admin). The former student and signatory report endpoints were removed, so all exports now come from the admin/OSAAR Reports & Export page.

Page / feature → objects accessed

Page / feature	Objects read or written	Patterns
Student dashboard	ORGANIZATION, EVENT, SUBMISSION, ANNOUNCEMENT, COLLAB POINTER, POSTEVAL	1, 2, 3, 4, 11, 13, 29
Submission tracker / detail	SUBMISSION, NOTIFICATION, SIGNATORY (by copied id)	5, 6
New / edit submission (SAAF)	SUBMISSION, COLLAB POINTER, ORGANIZATION (directory), CAMPUS, RESERVABLE, BOOKING (holds + `booking_refs`)	12, 15, 19, 20, 21, 23, 24
Signatory desk (inbox)	SUBMISSION, SIGNATORY	7, 9
Signatory review (approve/return/deny/classify)	SUBMISSION, NOTIFICATION, BOOKING (release holds on deny/return; approve holds on CDM approve), RESERVABLE	6, 16, 24, 39
Admin dashboard (submissions)	ORGANIZATION, SUBMISSION	10
Admin → Organizations	ORGANIZATION, SIGNATORY	11, 12, 17
Admin → Signatories	SIGNATORY, ORGANIZATION (desk guard)	8, 18
osaar → Campus	CAMPUS, RESERVABLE (delete guard)	19, 20, 26
cdm → Reservables (Add/View/Reserve)	CAMPUS, RESERVABLE, BOOKING	20, 21, 22, 23, 25, 26, 27
Student dashboard → request change (dialog)	SUBMISSION, RESERVATION_CHANGE, BOOKING (snapshot), NOTIFICATION	6, 28, 29
cdm → Reservations (resolve cancel / reschedule / deny)	RESERVATION_CHANGE, SUBMISSION, BOOKING, RESERVABLE, CAMPUS, NOTIFICATION	20, 21, 23, 24, 30, 31
About / Announcements	ANNOUNCEMENT	13, 14
Student → Post-evals (entry + wizard)	SUBMISSION, POSTEVAL, EVENT children (MEMBER#/EVALUATION#)	28, 31, 32, 33
Student dashboard → Post-eval panel	POSTEVAL	29
OSAAR → Post-evals (queue + review)	POSTEVAL, SUBMISSION (finish on approve)	30, 31, 32, 34
Auto-cancel on window lapse (Streams consumer, not a page)	CANCELWATCH (sentinel create/delete), SUBMISSION, POSTEVAL (re-check), NOTIFICATION	40, 41
Arcus student → My events / form	SUBMISSION, attendance child, STUDENT profile, full/slim EVALUATION records	35, 37

Page / feature	Objects read or written	Patterns
Arcus officer → Event workspace / export	SUBMISSION, POSTEVAL, attendance/evaluation children, full EVALUATION records	32, 36, 38